University of Konstanz
Department Mathematics and Statistics
Jun-Prof. Dr. B. Azmi
M. Kartmann

Summer semester 2025

Optimization IV - PDE-constrained Optimization

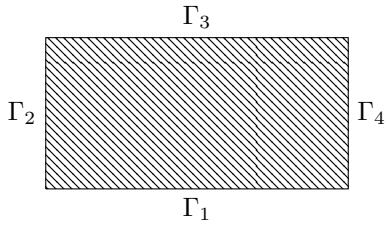
Exercise Sheet 1

Submission Deadline: April 29th 00:00h

On this sheet we will recall some theory of linear and coercive elliptic problems in Exercise 1.1 and create a corresponding finite element model in Exercise 1.2. If you need help with anything, just ask!

Exercise 1.0. (0 Points)
Create a virtual environment with the name `opti4` and install FEniCS (legacy Version 2019.1.0) or FEniCSx in this environment. Since the focus of this lecture is the optimization of PDEs and not the Finite Element discretization of PDEs both versions are suitable. For Windows users: create a Docker container for FEniCS (see the FEniCS homepage). Reading the FEniCS Tutorial available via `https://fenicsproject.org/tutorial/` might be useful and example codes are given there.

Exercise 1.1. (Theory, 10 points + 3 bonus points)
In this this exercise we will recall some basic theory for linear, elliptic PDEs. Let $\Omega \subset \mathbb{R}^2$ be a non-empty, rectangular domain. Its boundary $\partial\Omega = \Gamma_G \cup \Gamma_N \cup \Gamma_R$ is partitioned into disjoint sets Γ_D, Γ_N,Γ_R and each set consist of a union of some of the four boundary segments Γ_i enumerated as:



On the boundary we prescribe the Dirichlet data $g_{\text{D}} : \Gamma_D \to \mathbb{R}$, the Neumann data $g_{\text{N}} : \Gamma_N \to \mathbb{R}$ and the Robin data $g_{\text{R}}, d_{\text{R}} : \Gamma_R \to \mathbb{R}$. Moreover, we consider the source $f : \Omega \to \mathbb{R}$, the diffusion and the reaction coefficient $\kappa, c : \Omega \to \mathbb{R}$. Given a control $u \in U = L^2(\Omega)$, we are interested in solving the state equation for the state $y = y(u) \in V$:

$$\begin{aligned} -\operatorname{div}(\kappa\nabla y) + cy &= f + u && \text{in } \Omega, \\ y &= g_{\text{D}} && \text{on } \Gamma_D, \\ \kappa\nabla y \cdot \nu &= g_{\text{N}} && \text{on } \Gamma_N, \\ \kappa\nabla y \cdot \nu + d_{\text{R}} y &= g_{\text{R}} && \text{on } \Gamma_R. \end{aligned} \tag{1}$$

For simplicity we set $g_{\text{D}} \equiv 0$ in the following.

(i) Identify an appropriate function space V of test and ansatz functions and derive the weak formulation of (1) in the form

$$a(y, v) = l(v) + b(u, v) \quad \forall v \in V. \tag{2}$$

Note that an equivalent formulation of (2) is given as

$$\mathcal{A}y = l + \mathcal{B}u \in V'. \tag{3}$$

for $\mathcal{A} : V \to V', y \mapsto a(y, \cdot)$, $\mathcal{B} : U \to V', u \mapsto b(u, \cdot)$ and $l \in V'$. Here V' denotes the dual space of V. State the neccessary regularity assumptions on the data $f, g_{\text{N}}, g_{\text{R}}, d_{\text{R}}, \kappa, c$ for the weak formulation to be well-defined.

(ii) Assume now $\kappa, c > 0$ a.e. in Ω. Use the Lax Milgram lemma, prove the existence of a unique weak solution $y = y(u) \in V$ of (1) for any given control $u \in U$. Bonus: what happens if we only assume $c \geq 0$?

(iii) Bonus: Now additionally assume $f, g_{\text{N}}, g_{\text{R}} \equiv 0$. Prove that in this case the solution operator $\mathcal{S} : U \to V, u \mapsto \mathcal{S}(u) = y(u)$ is linear and continuous.

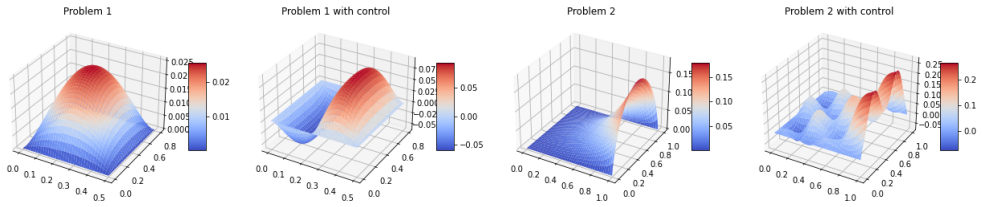


Figure 1: The states for the different problems.

Exercise 1.2. (Programming, 10 points)
In this exercise we will create an finite element model of (1) on our computer and discuss the numerical solution of its weak form (2) with FEniCS. We consider the case without Robin boundary conditions $\Gamma_R = \emptyset$ and a constant diffusion and reaction coefficient $\kappa, c \in \mathbb{R}$. Moreover recall that $g_{\text{D}} \equiv 0$. In this case we can write the bilinear form a as:

$$a(y, v) = \kappa a_1(y, v) + c a_2(y, v) \tag{4}$$

(i) Create an empty class `analytical_problem` that contains the lower left edge (a_1, b_1), and the upper right edge (a_2, b_2) of the domain $\Omega = [a_1, a_2] \times [b_1, b_2]$, the boundary parts Γ_D and Γ_N (identified by the indices Γ_i of the boundary segments to make them up), the data f, g_{N} as FEniCS `Expression` objects, the constants c, κ.

(ii) Write a python script that performs the following steps:

1. Generate a mesh of the domain using and plot it.
2. Create the discrete function space $V_h = \text{span}\{\varphi_1, ..., \varphi_N\} \subset V$ of first-order Lagrange elements using the command `FunctionSpace`. The degrees of freedom $N \in \mathbb{N}$ are defined through the mesh size h. We use the same function space for the control $U_h = V_h$. The discrete weak form is obtained via Galerkin projection of (5) onto V_h: given $u_h \in U_h$ find $y_h(u_h) \in V_h$ as the solution of

$$a(y_h, v_h) = l(v_h) + b(u_h, v_h) \quad \forall v_h \in V_h. \tag{5}$$

   Note that we can expand the solution as $y_h = \sum_{i=1}^{N} \mathbf{y}_i \varphi_i$ for a coefficient vector $\mathbf{y} \in \mathbb{R}^N$.
3. In FEniCS specify the (bi-) linear forms a_1, a_2, l and b corresponding to the weak formulation of (2) and the structure of a in (4). Further, specify the homogeneous dirichlet boundary conditions as an object `dirichlet_conditions`.
4. Use the function `assemble` and `dirichlet_conditions.apply` to obtain the discrete system matrices $A_1, A_2, B \in \mathbb{R}^{N \times N}$ and $L \in \mathbb{R}^N$, with which we can rewrite the discrete weak (5) form as

$$\underbrace{(\kappa A_1 + c A_2)}_{=:A} \mathbf{y} = L + B\mathbf{u} \in \mathbb{R}^N, \tag{6}$$

   where $\mathbf{y}, \mathbf{u} \in \mathbb{R}^N$ are the coefficient vectors of $y_h, u_h \in V_h$. Obtain the assembled discrete matrices as a numpy array (or even better as a sparse array).
5. Write a function `solution_operator` that takes as an input your discrete system matrices and a control $\mathbf{u} \in \mathbb{R}^N$ and computes the state $\mathbf{y} = \mathcal{S}(\mathbf{u}) \in \mathbb{R}^N$ as a solution of (6). The function should perform two steps: 1. apply the homogeneous dirichlet boundary conditions onto the control term $B\mathbf{u}$ (and all the other terms in (6) if not done already) and 2. solve the arising linear system (6) using the function `numpy.linalg.solve` (or e.g. `scipy.sparse.linalg.spsolve` in case you have sparse arrays).

If you are familiar with object oriented programming (`docs.python.org/3/tutorial/classes`), you can solve this exercise by creating a class `elliptic_model` that performs the steps described above.

(iii) Test your code on the following problems to get a feeling how the control influences the state. Define the control $u \in V$ as an `Expression` object and use the function `interpolate` to get a discrete function $u_h \in V_h$ and the corresponding coefficient vector $\mathbf{u} \in \mathbb{R}^N$. Plot the states in the domain Ω and compare them with Figure 1. Choose your mesh with a mesh size of about $h = 0.005$.

1. $\Omega = [0, 0.5] \times [0, 0.8]$, $\Gamma_D = \partial\Omega$, $f(x, y) = 1 - (x - 0.5)(y - 0.5)$, $\kappa = 1$, $c = 0$, $g_{\text{N}} \equiv 0$. Solve the problem for the controls $u \equiv 0$, $u(x, y) = -10\chi_{[0,0.25)\times[0,0.8]}(x, y) + 10\chi_{[0.25,0.5]\times[0,0.8]}(x, y)$.
2. $\Omega = [0, 1]^2$, $\Gamma_N = \Gamma_3 = \{(x, y) \in \Omega \,|\, x = 1\}$, $\Gamma_D = \partial\Omega \setminus \Gamma_N$, $f \equiv 0$, $g_{\text{N}}(x, y) = x^2$, $\kappa = c = 1$. Solve the problem for the controls $u = 0$, $u(x, y) = 50 \sin(4\pi x) \cos(4\pi y)$.